Banking System

Project Title

Banking System using Python (Deposit, Withdraw, Balance Management)

Introduction

The Banking System is a Command Line Interface (CLI)-based application developed in Python that simulates basic banking operations such as account creation, deposit, withdrawal, and balance inquiry.

This project demonstrates the implementation of **data handling, file storage, and modular programming** concepts using Python. It is designed to run entirely in a terminal environment without any graphical interface.

Objective

The primary objectives of this project are:

- To implement a real-world banking system using Python
- To demonstrate file handling using JSON
- To build a CLI-based interactive application
- To apply Object-Oriented Programming (OOP) concepts
- To simulate secure and structured financial transactions

Motivation

Banking systems are a fundamental part of everyday life. This project is motivated by the need to understand how basic banking operations work internally and how software systems manage financial data efficiently.

Problem Statement

Design and develop a CLI-based banking application that:

- Allows users to create accounts
- Supports deposit and withdrawal operations
- Maintains account balances persistently
- Handles invalid inputs and errors gracefully

Features

- Create new bank account
- Deposit money
- Withdraw money
- Check account balance
- Persistent storage using JSON file
- Error handling (invalid inputs, insufficient balance)
- Modular code structure (separation of concerns)

System Architecture

User (CLI Input)

↓

main.py (Interface Layer)

↓

bank.py (Business Logic)

↓

data.json (Storage Layer)

Project Structure

```
banking_system/

|— main.py       # CLI interface
```

```
|— bank.py       # Core banking logic
|— data.json     # Data storage file
|— README.md     # Project documentation
```

Technologies Used

- **Programming Language:** Python
- **Data Storage:** JSON file
- **Concepts Used:**
  - Object-Oriented Programming (OOP)
  - File Handling
  - Exception Handling
  - CLI Interaction

How It Works

1. Account Creation

- User enters name
- System checks if account exists
- If not, creates account with balance = 0

2. Deposit

- User enters amount
- System validates input
- Updates balance

3. Withdrawal

- Checks if sufficient balance exists
- Deducts amount if valid

4. Balance Inquiry

- Displays current balance

Data Storage Format

Data is stored in data.json as a dictionary:

```
{
   "Amit": 5000,
   "Rahul": 2000
}
```

How to Run the Project

Step 1: Install Python

Ensure Python is installed on your system.

Step 2: Clone Repository

```
git clone https://github.com/your-username/banking-system
cd banking-system
```

Step 3: Run the Application

```
python main.py
```

Sample Output

```
=== BANKING SYSTEM ===


1. Create Account
2. Deposit
3. Withdraw
4. Check Balance
5. Exit


Enter choice: 1
```

Enter your name: Amit

Account created successfully!

AI/ML Relevance

Although this project is rule-based, it lays the foundation for AI integration. Future enhancements can include:

- Spending pattern analysis
- Fraud detection using machine learning
- Smart financial recommendations
- Predictive balance forecasting

Future Enhancements

- PIN-based authentication system
- Transaction history tracking
- Database integration (SQLite/MySQL)
- Web or GUI interface
- Data visualization of spending habits
- AI-based financial assistant

Limitations

- No authentication/security (basic version)
- Stores data locally (not scalable)
- No multi-user concurrency handling
- No encryption of sensitive data

Testing & Validation

- Tested for:

    - Invalid inputs
    - Duplicate account creation
    - Insufficient balance withdrawal
    - File handling errors

Real-World Applicability

This project can be extended into:

- ATM simulation systems
- Personal finance trackers
- Banking backend prototypes
- Financial analytics tools

Key Learnings

- Practical implementation of file handling
- Structuring CLI-based applications
- Data persistence using JSON
- Writing modular and maintainable code

References

- Python Official Documentation
- JSON Documentation
- Basic Banking System Concepts